

Examen de fin de semestre SRCS Master 1 SAR

Jonathan Lejeune

Mai 2022

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Dans le code java demandé, vous ne gérerez pas les imports ni les exceptions **sauf si c'est explicitement demandé**
- Soyez synthétique et précis dans vos réponses
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.
- Lorsque vous ne savez pas répondre à une question, il est préférable de s'abstenir que de répondre une absurdité.

Exercice 1 – Questions de cours (7 points)

Question 1 1 point

Qu'est-ce que protobuf ?

Question 2 1 point

Qu'est-ce qu'un appel distant purement asynchrone ? Quelles garanties y a-t-il sur l'exécution de la fonction ? Ces garanties sont-elles indépendantes du protocole de transport sous-jacent (justifier) ?

Question 3 1 point

Donnez deux intérêts à utiliser des appels distants semi-synchrones.

Question 4 1 point

Que permet d'abstraire les EJB par rapport à RMI ?

Question 5 1 point

Quelles informations sont contenues dans la première ligne d'un message de requête HTTP ?
Même question sur un message de réponse ?

Question 6 1 point

Qu'est-ce que le type MIME ? Donner un exemple.

Question 7 1 point

Soit Alice qui envoie un message à Bob. Bob a-t-il moyen de détecter que le message a été corrompu par Oscar ? Si oui, expliquer comment, sinon expliquer pourquoi.

Exercice 2 – DHT (14 points)

Une table de hachage permet de stocker des objets dans une structure de données qui s'apparente à un tableau de listes. La taille du tableau est statiquement défini. Lors de l'ajout d'un objet, on calcule une empreinte de cet objet grâce à une fonction de hachage. Cette dernière renvoie un entier encodé sur un nombre de bits fixes. En calculant le modulo de ce nombre par rapport à la taille du tableau, il est ainsi possible de déterminer une case du tableau et d'ajouter l'objet à la liste correspondante. On applique le même raisonnement pour tester la présence d'un objet dans la table. En java ceci est implémenté par la classe `HashSet` ou bien `HashMap` si on considère une association clé-valeur (l'indexation se fait sur l'objet clé). Dans les deux cas, la fonction de hachage utilisée est la méthode `int hashCode()` (l'empreinte est donc sur 32 bits) de la classe `Object` qu'il est nécessaire de redéfinir dans la classe des objets que l'on souhaite indexer.

Question 1 1 point

Quelles pourraient être les conséquences d'une mauvaise fonction de hachage ?

Pour agréger la capacité de stockage de N nœuds, on peut implanter un protocole qui définit une table de hachage distribuée (dénommée DHT par la suite). Ici, un intervalle de l'ensemble des empreintes est sous la responsabilité d'un nœud. La figure 1 donne un exemple d'une DHT à 5 nœuds et sur une empreinte codée sur 8 bits : le nœud 0 stockera toutes les empreintes allant de 0 à 50, le nœud 1 de 51 à 101, etc.

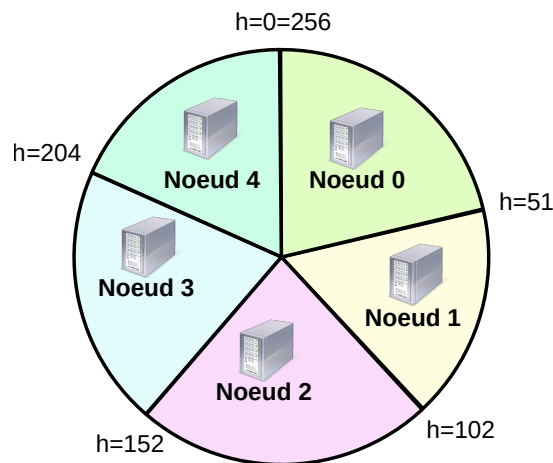


FIGURE 1 – Exemple de DHT

Dans le cadre de cet exercice, on souhaite implanter en java RMI une table associative distribuée (`Map<K,V>`) avec une DHT. Les méthodes offertes sont :

- `put` : associer une nouvelle valeur de type `V` à une clé de type `K`.
- `get` : retourner la valeur associée à une clé donnée. Si l'entrée n'existe pas dans la map alors `null` sera renvoyé.

Un client souhaitant utiliser la DHT peut le faire depuis n'importe quel nœud. Ainsi, si le nœud interrogé par le client ne contient pas la donnée concernée (ex : un client interroge le nœud 1 mais pour une clé ayant pour empreinte 230), c'est à la charge du nœud de transmettre la requête. La politique de retransmission dépendra de l'implantation que nous traiterons plus tard. On supposera par la suite que les nœuds ne peuvent pas tomber en panne et que le nombre de nœuds noté N est une constante.

Question 2 1 point

Donner l'interface java `MapDHTNode<K,V>` qui spécifie les services offerts par un nœud. Vous préciserez le cas échéant la contrainte de typage de `K` et `V`.

Question 3 1 point

L'implantation par défaut de la méthode `int hashCode()` retourne une valeur entière qui est dérivée de l'adresse mémoire de l'objet dans le tas. Dans le cadre de la DHT, que pourrait-il se passer si on ne redéfinit pas cette méthode? De quoi le résultat de cette méthode doit-il dépendre?

Par la suite, nous allons considérer que les nœuds ont sous leur responsabilité la même taille d'intervalle d'empreintes et qu'ils possèdent chacun un identifiant entier allant de 0 à $N - 1$. Nous supposons qu'il existe une valeur d'empreinte comprise entre 0 et E_{max} (où $E_{max} < 2^{31}$ pour éviter le dépassement de capacité des int). Ainsi les bornes de l'intervalle d'un nœud d'identifiant i est compris entre $i * E_{max}/N$ et $((i + 1) \bmod N) * E_{max}/N$.

Question 4 3 points

Proposer une classe `MapDHTNodeImpl` qui implante l'interface `MapDHTNode` :

- le stockage local des données se fera en mémoire vive, il n'est pas demandé d'utiliser des fichiers
- le constructeur prendra en paramètre N (entier), E_{max} (entier), l'identifiant du nœud courant (entier) et une référence sur un registry rmi qui permet de récupérer les références sur les autres nœuds.
- si le nœud courant reçoit une requête `get` ou `put` sur une clé dont il n'est pas responsable, il s'adressera directement au nœud responsable, attendra sa réponse et répondra au client.

Question 5 1½ points

Écrire une méthode statique `deployNewMapDHTNodeImpl` qui a exactement les mêmes arguments que le constructeur de `MapDHTNodeImpl`, qui l'instancie et déploie l'instance créée en tant qu'objet distant.

Question 6 1½ points

Le protocole de déploiement de la DHT consiste à démarrer parallèlement sur notre réseau de machines, 1 JVM dédiée au registry et N JVMs qui déploieront chacune un objet distant d'interface `MapDHTNode`. Proposer en quelques lignes de texte une solution qui assure que toute consultation du registry par un nœud a pour avoir la référence distante d'un nœud b se fera toujours après que b se soit enregistré dans le registry. Énoncer les avantages et les inconvénients de votre solution.

Nous souhaitons maintenant répliquer les données X fois ($X < N$). On considère alors un réplica primaire, la version de la donnée qui est stockée sur le nœud qui en serait responsable sans réplification et $X - 1$ réplicas secondaires qui sont stockés sur les $X - 1$ nœuds successeurs du nœud stockant le réplica primaire. Dans l'exemple de la figure 1, si on considère que $X = 3$ alors toute donnée stockée primairement sur le nœud 0 existera également secondairement sur les nœuds 1 et 2 (ce raisonnement est aussi valable pour les nœuds 1 à 4).

Question 7 1 point

Donner un avantage et un inconvénient de répliquer les données.

Question 8 3 points

Proposer une nouvelle implantation de `MapDHTNode` qui respecte les spécifications suivantes :

- en cas de requête de lecture (`get`), le nœud peut répondre directement au client si il possède un des réplicas de la donnée (primaire ou secondaire)
- en cas de requête d'écriture (`put`), le nœud doit retransmettre la requête au nœud possédant le réplica primaire. Lorsque le primaire reçoit une requête d'écriture de la part d'un de ses pairs ou d'un client, il met à jour son réplica et requête **parallèlement** l'ensemble des réplicas secondaires pour les mettre à jour. La réponse au client ne doit être faite une fois que le primaire est certain que tous les réplicas secondaires ont été mis à jour.

On supposera que la valeur **X** est passée en paramètre du constructeur. Votre solution peut si besoin définir de nouvelles classes ou interfaces. En revanche, l'interface **MapDHTNode** ne doit pas changer.

N.B. : Pour gagner du temps, il est possible d'écrire en commentaire le code que vous gardez de la classe précédente.

Question 9 1 point

Quelle(s) modification(s) faudrait-il faire sur le protocole pour que les noeuds hébergeant des réplicas secondaires soient assurés que toute mise à jour soit uniquement à l'initiative du nœud possédant le réplica primaire ?